Midterm Revision

Week 1

Core Question: How do independent processes communicate

Distributed System

collection of independent computers that communicate over a network and work together as one system.

Pros	Meaning	Cons	Meaning
Resource Sharing	Making things available through multiple hosts	Latency	May need to cross software and networks
Scalability	Being able to increase resources	Uncertain Ordering	Message delay and may be unordered
Availability	Still able to run even when something fails	Partial Failure	Component can be dysfunctional while others still work
Geographic reach	Being able to reach users	Security Exposure	Communication crosses trust
Specialization	Different components having different functions	Operational complexity	deployment monitoring and configuraiton need care
Collaboration	Being able to share content with others	Concurrency	Clients may overlap at the same time

IPC

mechanisms that allow two or more processes to exchange data and coordinate their actions

A process is a program that is currently running

A **client** sends a request to a **server** using a **protocol**. The server provides a service through an **endpoint**. A **socket** enables communication, and a **connection** links the communicating endpoints.

Term	Short definition
Host	A computer or device connected to a network.
Process	A program that is currently running.
Thread	A unit of execution within a process.
Client	A program or device that requests a service or resource.
Server	A program or computer that responds to client requests.
Service	A function or resource provided to other programs or devices.
Endpoint	A specific address where a service can be accessed.
Socket	A communication endpoint used to send and receive data.
Connection	A communication link established between two endpoints.
Protocol	A set of rules that defines how devices or programs communicate.

Pattern	Everyday example	Message route and important trade-off
Client–server / multi-tier	School portal	Browser → API → relational database. Roles and storage remain separate.
Work queue	Receipt email after a purchase	Checkout → queued job → email worker. Acceptance of a job is not proof the email was delivered.
Publish–subscribe	Notifications to interested systems	Publisher → topic/broker → several subscribers. Each consumer may progress or fail independently.
Fan-out / fan-in	Travel search	Search service calls multiple providers and combines results. Latency and partial-result rules matter.
Replicated service / load balancer	Busy ticket launch	A gateway distributes requests across instances. Shared state must still be coordinated.
Peer-to-peer / streaming	Peer file exchange / live updates	Peers may both provide and consume; streams need framing, lifecycle and backpressure policies.

Why do timeouts happen?

- The server is overloaded.
- The internet connection is slow or interrupted.
- The database query takes too long.
- The server is unavailable or not responding.

Term	Role / definition
Names	Human-readable identifiers used to identify resources or hosts, such as `example.com`.
IP addresses	Identify a host or network interface so data can be routed to the correct destination.
Port numbers	Identify a specific application or service on a host, such as port `443` for HTTPS.
TCP	A transport protocol that provides reliable, ordered delivery of data.
UDP	A transport protocol that sends datagrams without guaranteeing delivery or order.
Application-level message framing	Defines how an application identifies the beginning and end of each message in a stream of data.

Week 2

Core Question: What does an HTTP exchange mean?

- Internet: A global network connecting computers and devices.
- Web: A collection of websites and web pages accessed through the Internet.
- Web application: A website that lets users perform tasks or interact with data.

CDN (Content Delivery Network): A network of servers distributed around the world that delivers website content to users faster by using servers closer to them.

TLS (Transport Layer Security): A protocol that protects data sent over a network by encrypting it.

URIs

URI (Uniform Resource Identifier): A string used to identify a resource on the Internet.

Example: `https://example.com:443/products?id=5#details`

- Scheme: `https` — how to access the resource.
- Host: `example.com` — the server's domain name.
- Port: `443` — the communication endpoint on the server.
- Path: `/products` — the resource's location.
- Query: `id=5` — additional parameters.
- Fragment: `details` — a specific section of the resource.

Term	Full name	Purpose	Example / note
URI	Uniform Resource Identifier	Identifies a resource.	General category that includes URLs and URNs.
URL	Uniform Resource Locator	Identifies a resource and tells you where to find it.	`https://example.com/page` — tells you where to access a page.
URN	Uniform Resource Name	Identifies a resource by its name, regardless of its location.	`urn:isbn:9780131103627` — identifies a book by its ISBN.

HTTP (Hypertext Transfer Protocol): A protocol that allows web browsers and servers to communicate and exchange data.
Example: When you visit a website, your browser uses HTTP to request a web page from a server.

Requests/Responses

Message	Definition	Example
HTTP Request	A message sent by a client (such as a browser) to a server asking for data or an action.	A browser requests a webpage.
HTTP Response	A message sent by the server back to the client with the result.	The server sends the requested webpage.

```
GET /courses/420-511?term=F26 HTTP/1.1
Host: portal.example.edu
Accept: text/html
User-Agent: ExampleBrowser/1.0
Cookie: session=abc123
```

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: private, max-age=60
ETag: "course-v17"
```

Methods

idempotent: repetition of the same request that end up with the same result

Method	Purpose	Idempotent?
GET	Retrieve data.	y
POST	Create or send data.	n by default
PUT	Replace existing data.	y
PATCH	Update part of existing data.	n guaranteed
DELETE	Delete data.	y
HEAD	Retrieve headers without the response body.	y
OPTIONS	Find out which methods a server supports.	y

HTTP States (Status Codes)

Status codes: Tell the client whether a request succeeded or failed.

Example: `200 OK` means the request succeeded, while `404 Not Found` means the resource could not be found.

Code	Meaning	Example
1xx	Informational	Request processing continues
2xx	Success	200 OK
3xx	Redirection	301 Moved Permanently
4xx	Client error	404 Not Found
5xx	Server error	500 Internal Server Error

HTTP Headers

Headers: Provide extra information about an HTTP request or response.

Example: `Content-Type: application/json` tells the server that the data is in JSON format.

Caches

Cache: A temporary storage location that keeps copies of data so they can be accessed faster later.
Example: Your browser stores images from a website in its cache so they load faster when you visit the site again.
Benefits: Faster loading times, less network traffic, and reduced server workload.

HTTPS

HTTPS (Hypertext Transfer Protocol Secure): A secure version of HTTP that uses TLS to encrypt data exchanged between a browser and a server.

How it differs from HTTP:

HTTP	HTTPS
Data is not encrypted by HTTP itself.	Data is encrypted using TLS.
Data can be easier to intercept and read.	Data is protected from being read by outsiders.

HTTP	HTTPS
Uses port 80 by default.	Uses port 443 by default.
Does not provide TLS security.	Provides encryption, data integrity, and server authentication through TLS.

Example: When you log in to a website, HTTPS helps protect your password while it travels between your browser and the server.

Week 3

Core Question: How should a Web service be designed?

API (Application Programming Interface): A set of rules that allows different software applications to communicate and exchange data.
Example: A weather app uses a weather API to retrieve current weather information from a server.

REST (Representational State Transfer): A set of architectural principles used to design web APIs that allow applications to communicate over HTTP.

RESTful API: An API that follows REST principles to let applications access and manage resources using HTTP methods.

Example: `GET /books` retrieves books, `POST /books` creates a book, and `DELETE /books/1` deletes book 1.

Why it is used: It makes communication between clients and servers simple, organized, and standardized.

Example: A library application uses a REST API to retrieve books with `GET /books` or add a book with `POST /books`.

Resources

Resource: An object, collection, relationships or piece of data that an API manages and allows clients to access or modify.

Example: In a library API, a book is a resource, accessed through `/books` or `/books/1`.

Representations

Representation: A format used to describe or transmit a resource's data, such as JSON or XML.

Example: A book resource can be represented as JSON: `{"title": "Dune", "author": "Frank Herbert"}`

Constraints

REST Constraints: Rules that guide how a RESTful API should be designed.

Constraint	Simple definition
Client-Server	The client and server have separate responsibilities.
Stateless	Each request contains the information needed to process it.
Cacheable	Responses can be stored and reused when appropriate.
Uniform Interface	Resources are accessed in a consistent, standardized way.
Layered System	Communication can pass through intermediaries, such as proxies.
Code on Demand (optional)	The server can send executable code to the client.

Remember: REST constraints make APIs more consistent, scalable, and easier to maintain.

Uniform Interface

Uniform Interface: A REST constraint that ensures clients interact with resources in a consistent and standardized way.

Example: An API uses `GET /books` to retrieve books and `DELETE /books/1` to delete a specific book.

Subconstraint	Simple definition
Identification of Resources	Each resource has a unique identifier, such as a URL.
Manipulation of Resources Through Representations	Clients use representations, such as JSON, to modify resources.

Subconstraint	Simple definition
Self-Descriptive Messages	Each message contains enough information to explain how it should be processed.
Hypermedia as the Engine of Application State (HATEOAS)	Clients navigate application state through links provided by the server.

Maturity

Richardson Maturity Model: A model that measures how closely an API follows REST principles, using four levels.

Level	Name	Simple definition
Level 0	The Swamp of POX	Uses HTTP mainly as a transport system, often with one endpoint for everything.
Level 1	Resources	Uses separate endpoints for different resources.
Level 2	HTTP Methods	Uses HTTP methods and status codes correctly, such as `GET`, `POST`, and `404`.
Level 3	Hypermedia Controls	Includes links in responses to guide clients toward available actions.

Example: A book API progresses from sending every request to one endpoint (Level 0) to using resource URLs, proper HTTP methods, and links to related actions (Level 3).

Remember: Level 0 = one endpoint; Level 1 = resources; Level 2 = HTTP methods; Level 3 = hypermedia links.

Week 4

Core Question: How do contract and code agree?

OpenAPI

OpenAPI: A standard for describing an API so developers can understand how to use it without reading its source code.
Example: An OpenAPI document describes available endpoints, HTTP methods, parameters, and responses for a library API.

```yaml
openapi: 3.0.0
info:
  title: Library API
  version: 1.0.0

paths:
  /books:
    get:
      summary: Get all books
      responses:
        '200':
          description: List of books retrieved successfully

    post:
      summary: Add a new book
      responses:
        '201':
          description: Book created successfully

  /books/{id}:
    get:
      summary: Get a book by ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Book found
        '404':
          description: Book not found
```

Compatibility

API change is compatible if existing clients still work according to the OpenAPI documentation, regardless of how small the code change is.

Example:

- Compatible: Adding a new optional field to a response.
- Breaking: Removing a documented field that existing clients rely on.

Remember: Compatibility depends on how the change affects users of the API, not how much code was changed.

API Versioning: Assigning versions to an API to manage changes while maintaining compatibility with existing clients.

Example: `/api/v1/books` and `/api/v2/books` allow clients to use different versions of a library API.

Remember: Minor changes can often keep the same version, while breaking changes may require a new version.

Routes/Controllers

Controller: The part of an application that receives requests, processes them, and determines what response to send.

Example: In a library API, a book controller handles requests like `GET /books` to retrieve books.

Front Controller: A central controller that receives all incoming requests and directs them to the appropriate controller.

Example: A request to `/books` goes through the front controller, which directs it to the `BookController`.

Remember: The front controller routes requests; resource controllers handle specific tasks.

Example: Library API

Controller	Responsibility
`BookController`	Create, retrieve, update, and delete books.
`AuthorController`	Manage authors.
`UserController`	Manage users.
`LoanController`	Manage borrowed and returned books.

Routes: Rules that connect a URL and HTTP method to the code that handles the request.

Example:

- `GET /books` → Retrieve all books.
- `POST /books` → Create a new book.
- `GET /books/1` → Retrieve book 1.
- `DELETE /books/1` → Delete book 1.

PDO

PDO (PHP Data Objects): A PHP feature that allows applications to connect to databases and execute SQL queries.
Example: A PHP library API uses PDO to retrieve books from a MySQL database.
Remember: PDO connects PHP to the database and helps execute queries securely using prepared statements.

Relational Integrity

Relational Integrity: Rules that ensure relationships between tables in a database remain valid and consistent.

Example: A book's `author_id` must reference an existing author in the `authors` table, using a foreign key.

Remember: Relational integrity prevents invalid relationships between database tables.

Transactions

Database Transactions: A group of database operations treated as one unit. Either all operations succeed, or the changes are rolled back.

Example: Transferring money between accounts requires subtracting money from one account and adding it to another. Both operations must succeed together.

Remember:

- `beginTransaction()` — Start a transaction.
- `commit()` — Save the changes.
- `rollBack()` — Undo the changes.

Week 5

Core Question: How do shared policy and collections work?

Middleware: Code that runs between receiving a request and sending a response, often to check authentication, validate data, or log requests.
Example: Middleware checks whether a user is logged in before allowing them to access a protected API endpoint.

Middleware Nesting

Middleware Nesting: The order in which multiple middleware functions run, passing a request through each one before reaching the route handler.
Example: A request passes through logging middleware, then authentication middleware, then the controller.
Remember: Middleware runs in a specific order, and each layer can process the request or stop it.

Validation

Validation: Checking that user input meets the required rules before processing or storing it.
Example: A web API checks that a book's title is not empty and that its ID is a valid number before saving it to the database.

Filtering

Filtering: Selecting only the data that matches specific conditions.

Example: A books API uses `GET /books?author=Frank Herbert` to return only books written by Frank Herbert.

Total Ordering

Total Ordering: Arranging all records in a consistent order, with a clear way to break ties.
Example: Sorting books by title, then by ID when two books have the same title.
Remember: A unique tie-breaker, such as an ID, ensures records always have a predictable order, which is especially important for pagination.

Pages

Pagination: Dividing a large set of data into smaller pages to make it easier to retrieve and display.

Example: `GET /books?page=2` retrieves the second page of books instead of returning every book at once.

Offset: The number of records to skip before retrieving data from a database.

Example: `LIMIT 10 OFFSET 20` skips the first 20 records and retrieves the next 10.

```
offset = (page − 1) × perPage = (2 − 1) × 2 = 2.
```

Remember: `LIMIT` controls how many records you retrieve; `OFFSET` controls how many you skip.

Links

Links (in REST APIs): URLs included in API responses that let clients access related resources or discover available actions.

Example: A book response includes a link to `/books/1/author` to access the book's author.

Remember: Links help clients navigate an API without having to guess endpoint URLs.

Cursors

Cursor: A marker that identifies where to continue retrieving data, commonly used for pagination.
Example: An API returns 10 books and a cursor. The client sends that cursor in its next request to retrieve the next set of books.
Remember: Unlike offset pagination, cursor pagination uses a marker to continue from the last position.

Week 6

Core Question: How are results represented and failures diagnosed?

JSON/XML/YAML

JSON (JavaScript Object Notation): A lightweight data format used to exchange data between applications.

Example:

```
{
  "title": "Dune",
  "author": "Frank Herbert",
  "year": 1965
}
```

Remember: JSON stores data using key-value pairs and is commonly used by REST APIs to send and receive data.

use application/json

XML (Extensible Markup Language): A text format that uses tags to store and exchange structured data.

Example:

```
<book>
  <title>Dune</title>
  <author>Frank Herbert</author>
```

```
  <year>1965</year>
</book>
```

Remember: XML uses tags to organize data, while JSON uses key-value pairs.

application/xml

YAML (YAML Ain't Markup Language): A human-readable data format used to store configuration and structured data.

Example:

```
title: Dune
author: Frank Herbert
year: 1965
```

Remember: YAML uses indentation and key-value pairs to organize data. It is commonly used for configuration files and OpenAPI documentation.

application/yaml

Feature	JSON	XML	YAML
Structure	Key-value pairs	Tags	Indentation and key-value pairs
Readability	Easy to read	More verbose	Very easy to read
Syntax	`{ }` and `[ ]`	`<tags>`	Indentation
Main use	APIs and data exchange	Data exchange and documents	Configuration files
Comments	Not supported	Supported	Supported

Remember:

- JSON: Common for REST APIs.
- XML: Uses tags to organize data.
- YAML: Uses indentation and is common in configuration files.

Negotiation

Content Negotiation: A process where a client and server agree on the format of the data being exchanged.

Example: A client sends `Accept: application/json` to request a JSON response from the server.

Remember: `Accept` specifies the format the client wants, while `Content-Type` specifies the format of the data being sent.

Safe Errors

Safe Errors: Error messages that explain what went wrong without exposing sensitive information about the system.

Example: Returning `500 Internal Server Error` with `"An unexpected error occurred"` instead of revealing database passwords or SQL queries.

Remember: Give users useful error messages without exposing sensitive technical details.

Error Handling as an API Contract: Defining consistent error responses so clients know what errors mean and how to handle them.

Example: An API returns `404 Not Found` with `{"error": "Book not found"}` when a requested book does not exist.

Remember: Documenting errors helps clients predict and handle failures consistently.

Correlation

Correlation ID: A unique identifier used to track a request across different services, making it easier to trace and troubleshoot errors.
Example: If a request fails while passing through multiple servers, the correlation ID helps developers find all logs related to that request.
Remember: A correlation ID connects related logs and events to the same request.

Structured Logs

Logging and Operational Evidence: Recording information about an application's activity to monitor its behavior, troubleshoot errors, and investigate problems.
Example: A server logs requests, response status codes, timestamps, and

correlation IDs to help developers find the cause of an API failure.
Remember: Logs provide evidence of what happened in an application.

Structured Logs: Logs recorded in a consistent format, such as JSON, so they can be easily searched, filtered, and analyzed.

Example:

```
{
  "status": 404,
  "method": "GET",
  "path": "/books/5"
}
```

Remember: Structured logs organize information into fields, making it easier to find and troubleshoot errors.